
Wish: A Wide Hash Function Family

Version 1.0

Shiyao Chen¹ Wenjie Nan² Meiqin Wang¹ Tianyu Zhang²

¹School of Cyber Science and Technology,
Shandong University, Qingdao, China
{shiyao.chen, mqwang}@sdu.edu.cn

²School of Physical and Mathematical Sciences,
Nanyang Technological University, Singapore
{wenjie006, tianyu005}@e.ntu.edu.sg

June 30, 2026

Contents

1	Introduction	2
2	Notation and Conventions	3
3	Algorithm Description	4
3.1	Parameters	4
3.2	State Representation	4
3.3	The Permutation	4
3.3.1	The Mixing Layer	5
3.3.2	The Tweak	5
3.3.3	The Round Constants	5
3.4	Mode: Sponge with Feed-Forward	6
3.5	Complete Algorithm	7
4	Design Rationale	7
4.1	Number of AES rounds per step	7
4.2	Choice of Mix	8
4.2.1	Desired Properties	9
4.2.2	Structure: A Cooley-Tukey FFT Butterfly of Interleaves	9
4.2.3	Final Selection for Mix	10
4.3	Tweakable permutations	10
5	Security Analysis	13
5.1	Security Claims (Classical and Quantum)	13
5.2	Resistance to Generic Attacks	13
5.3	Cryptanalysis	14
5.3.1	Statistical Randomness	14
5.3.2	Differential and Linear Attacks	14
5.3.3	Rebound Attacks	15
5.3.4	Algebraic and Integral Attacks	15
5.3.5	Meet-in-the-Middle Attacks	16
6	Performance	17
6.1	Software Performance	17
6.2	Hardware Performance	18

1 Introduction

Wish is a cryptographic hash function submitted in response to the NICCS call for proposals for the Next-generation Commercial Cryptographic Algorithms Program (NGCC)¹. The Wish hash family supports two output digest lengths: the 512-bit variant **Wish-512** and the 1024-bit variant **Wish-1024**.

On a high level, **Wish** defines an AES-based permutation family **Wish-P** and instantiate it with a tweakable variant of the sponge-with-feed-forward mode **Sponge-F** [GHJ⁺25]. The main features are:

- **High and Scalable Software Performance.** **Wish** takes 128-bit registers as the atomic unit of operation. One step of **Wish-P** applies two consecutive AES rounds to each register in parallel, followed by a sequence of 2-register interleave operations that achieve cross-register mixing. Both the AES round functions and the interleave operations are native instructions supported by modern architectures, for instance `aesenc`, `unpackhi/unpacklo` on x86-64, and `vaeseq/vaesmcq`, `vzip1q/vzip2q` on AArch64. Consequently, **Wish** achieves high software performance while retaining broad compatibility across modern CPUs. Moreover, the design of **Wish** is not tailored to the pipeline structure of any specific CPU architecture. Instead, its computation exhibits high parallelizability, and its performance scales with the resources the hardware provides, making the design forward-looking with respect to evolving ISAs and wider execution back-ends.
- **Tweakable Sponge-with-Feed-Forward Mode.** **Sponge-F** [GHJ⁺25] modifies the standard sponge construction [BDPA08] by feeding the c -bit capacity part of each permutation input forward into its output. This yields tighter security reductions and improved provable bounds based on suitable correlation-intractability properties of the underlying permutation. However, for a **Sponge-F** hash function with h -bit digest length and maximum $(2^{64} - 1)$ -bit input length, to maintain h -bit second-preimage security requires at least $(h + 64)$ -bit capacity. This reduces the effective rate for a fixed permutation width thus the absorption speed. In **Wish**, we apply a tweakable variant of **Sponge-F**, where we encode the block counter as public tweak τ and operate over a family of permutations **Wish-P** _{τ} . The innovative use of tweak provides clean domain separation among absorption positions without consuming rate or capacity bits, and thereby avoiding a second-preimage loss proportional to the message length.
- **Flexibility.** At the hash-function level, **Wish** inherits the flexibility of the sponge and duplex frameworks and can be easily be adapted to variable-output-length and duplex-style constructions, supporting a broad range of cryptographic applications, including tree hashing, key derivation, pseudo-random bit generation, stream encryption, message authentication code and authenticated encryption. At the primitive level, the **Wish-P** _{τ} family can also serve as a building block in other permutation-based constructions. The available permutation widths can be combined with different rate–capacity allocations and tweak formats to meet different security–efficiency requirements. When explicit domain separation is unnecessary, fixing the tweak to a public constant yields an ordinary untweaked permutation.

¹https://www.nNICCS.org.cn/nNICCS/Notice/pc/content/content_1975892908773478400.htm

2 Notation and Conventions

Throughout this document, all indices are zero-based. Subscripts on mode-level variables denote *iteration indices*: M_i is the i -th message block, and S_i , R_i , C_i denote the internal state and its rate and capacity parts after i permutation calls. Superscripts denote *iterated operations*: Step^j is the j -th step of the permutation. Individual bytes of the state are indexed in array form, written $S[\cdot]$.

Function composition is applied right to left: $(f \circ g)(x) = f(g(x))$, *i.e.*, in $f \circ g$ the map g is applied first.

For a byte permutation σ on $\{0, \dots, n/8 - 1\}$, we define its action on the state in array form $\sigma(S) : \sigma(S)[i] = S[\sigma(i)]$, *i.e.*, output byte i is taken from input position $\sigma(i)$.

We also denote an AES round as a sequence of $\text{AddRoundKey} \circ \text{MixColumns} \circ \text{ShiftRows} \circ \text{SubBytes}$, and half an AES round as $\text{ShiftRows} \circ \text{SubBytes}$.

Table 1: Notation used throughout this document.

Symbol	Meaning
$\oplus$	Bitwise XOR
$\parallel$	Concatenation
$\text{left}_k(X)$	Leftmost k bits of X
$\text{right}_k(X)$	Rightmost k bits of X
SubBytes	SubBytes Operator of AES
ShiftRows	ShiftRows Operator of AES
MixColumns	MixColumns Operator of AES
AddRoundKey	AddRoundKey Operator of AES
AddConsts	AddRoundConst Operator of AES
Wish-P_τ	The Wish permutation with tweak τ
Stepj	The j -th step of Wish-P_τ
t	Number of steps of Wish-P_τ
b	Number of padded message blocks
n	Width in bits of the internal state
ℓ	Number of AES lanes of the internal state; $\ell = n/128$
r	Width in bits of the rate part
c	Width in bits of the capacity part
h	Hash output length in bits
M	The input message; $M \in \{0, 1\}^*$
M_i	The i -th padded message block
S_i	The internal state after i permutation calls
L_k	The k -th AES lane of the state; $L_k \in \{0, 1\}^{128}$
R_i	The rate part of S_i
C_i	The capacity part of S_i
τ_i	The tweak of the i -th permutation call
θ	The final-block separation constant; $\theta \in \{0, 1\}^c$

Unless otherwise stated, all multi-byte strings are processed in big-endian order. The permutation **Wish- P_τ** operates on n bits viewed as ℓ full 128-bit AES lanes; the AES round function is applied identically to all lanes in parallel.

3 Algorithm Description

3.1 Parameters

Wish is a wide, robust, and flexible hash function family supporting input messages of length at most $2^{64} - 1$ bits. The design of Wish combines an AES-based permutation Wish-P with a tweakable variant of the Sponge-with-feed-forward (Sponge-F) mode [GHJ⁺25]. The concrete parameters of Wish-512 and Wish-1024 in Table 2.

Table 2: Parameters of the Wish family.

Instance	n	ℓ	t	r	c	h
Wish-512	1024	8	9	512	512	512
Wish-1024	2048	16	12	1024	1024	1024

3.2 State Representation

The internal state S is organized as an array of $n/8$ bytes. Every 4 consecutive bytes form a *column*, and every 4 consecutive columns form a *lane*. A lane is therefore an array of 16 bytes and can be viewed as a 4×4 byte matrix following the AES ordering convention: the byte at row i and column j of lane k (with $0 \leq i, j \leq 3$) is the byte $(i + 4j)$ of L_k , equivalently the byte $(i + 4j + 16k)$ of S . Below is the matrix form of lane k .

$$L_k = \begin{bmatrix} S[0 + 16k] & S[4 + 16k] & S[8 + 16k] & S[12 + 16k] \\ S[1 + 16k] & S[5 + 16k] & S[9 + 16k] & S[13 + 16k] \\ S[2 + 16k] & S[6 + 16k] & S[10 + 16k] & S[14 + 16k] \\ S[3 + 16k] & S[7 + 16k] & S[11 + 16k] & S[15 + 16k] \end{bmatrix} \quad (1)$$

The rate part R occupies the first half of the state and the capacity part C the second half, *i.e.*, $R = L_0 \parallel \dots \parallel L_{\ell/2-1}$ and $C = L_{\ell/2} \parallel \dots \parallel L_{\ell-1}$. In this paper, we refer to a *column vector* as an array of 4 bytes and a *lane vector* as an array of 16 bytes.

3.3 The Permutation

The permutation $\text{Wish-P}_\tau : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a family of AES-based permutations with t steps, parameterized by a lane vector tweak τ :

$$\text{Wish-P}_\tau = \text{Step}^{t-1} \circ \dots \circ \text{Step}^1 \circ \text{Step}^0.$$

We let $\text{AESR} := \text{MixColumns} \circ \text{ShiftRows} \circ \text{SubBytes}$ denote one key-less AES round function, applied in parallel to each lane of the state. Step j is composed of

$$\text{Step}^j = \text{Mix} \circ \text{AddConsts}^{2j+1} \circ \text{AESR}^{2j+1} \circ \text{AddConsts}^{2j} \circ \text{AESR}^{2j} \circ \text{AddTweak},$$

where Mix is a byte permutation of the whole internal state (defined in Section 3.3.1), AddTweak adds the tweak (defined in Section 3.3.2), AddConsts^{2j} and AddConsts^{2j+1} add the round constants of step j (defined in Section 3.3.3).

3.3.1 The Mixing Layer

The mixing layer **Mix** is a byte permutation that achieves cross-lane diffusion. Its design rationale and implementation is detailed in [Section 4.2](#). Here we only give a specification of **Mix** in the array form: the i -th entry, read in row-major order, is the input position from which output byte i is drawn².

$$\text{Mix}_{1024} = \begin{bmatrix} 0 & 16 & 32 & 48 & 1 & 17 & 33 & 49 & 64 & 80 & 96 & 112 & 65 & 81 & 97 & 113 \\ 8 & 24 & 40 & 56 & 9 & 25 & 41 & 57 & 72 & 88 & 104 & 120 & 73 & 89 & 105 & 121 \\ 4 & 20 & 36 & 52 & 5 & 21 & 37 & 53 & 68 & 84 & 100 & 116 & 69 & 85 & 101 & 117 \\ 12 & 28 & 44 & 60 & 13 & 29 & 45 & 61 & 76 & 92 & 108 & 124 & 77 & 93 & 109 & 125 \\ 2 & 18 & 34 & 50 & 3 & 19 & 35 & 51 & 66 & 82 & 98 & 114 & 67 & 83 & 99 & 115 \\ 10 & 26 & 42 & 58 & 11 & 27 & 43 & 59 & 74 & 90 & 106 & 122 & 75 & 91 & 107 & 123 \\ 6 & 22 & 38 & 54 & 7 & 23 & 39 & 55 & 70 & 86 & 102 & 118 & 71 & 87 & 103 & 119 \\ 14 & 30 & 46 & 62 & 15 & 31 & 47 & 63 & 78 & 94 & 110 & 126 & 79 & 95 & 111 & 127 \end{bmatrix}, \quad (2)$$

$$\text{Mix}_{2048} = \begin{bmatrix} 0 & 16 & 32 & 48 & 64 & 80 & 96 & 112 & 128 & 144 & 160 & 176 & 192 & 208 & 224 & 240 \\ 8 & 24 & 40 & 56 & 72 & 88 & 104 & 120 & 136 & 152 & 168 & 184 & 200 & 216 & 232 & 248 \\ 4 & 20 & 36 & 52 & 68 & 84 & 100 & 116 & 132 & 148 & 164 & 180 & 196 & 212 & 228 & 244 \\ 12 & 28 & 44 & 60 & 76 & 92 & 108 & 124 & 140 & 156 & 172 & 188 & 204 & 220 & 236 & 252 \\ 2 & 18 & 34 & 50 & 66 & 82 & 98 & 114 & 130 & 146 & 162 & 178 & 194 & 210 & 226 & 242 \\ 10 & 26 & 42 & 58 & 74 & 90 & 106 & 122 & 138 & 154 & 170 & 186 & 202 & 218 & 234 & 250 \\ 6 & 22 & 38 & 54 & 70 & 86 & 102 & 118 & 134 & 150 & 166 & 182 & 198 & 214 & 230 & 246 \\ 14 & 30 & 46 & 62 & 78 & 94 & 110 & 126 & 142 & 158 & 174 & 190 & 206 & 222 & 238 & 254 \\ 1 & 17 & 33 & 49 & 65 & 81 & 97 & 113 & 129 & 145 & 161 & 177 & 193 & 209 & 225 & 241 \\ 9 & 25 & 41 & 57 & 73 & 89 & 105 & 121 & 137 & 153 & 169 & 185 & 201 & 217 & 233 & 249 \\ 5 & 21 & 37 & 53 & 69 & 85 & 101 & 117 & 133 & 149 & 165 & 181 & 197 & 213 & 229 & 245 \\ 13 & 29 & 45 & 61 & 77 & 93 & 109 & 125 & 141 & 157 & 173 & 189 & 205 & 221 & 237 & 253 \\ 3 & 19 & 35 & 51 & 67 & 83 & 99 & 115 & 131 & 147 & 163 & 179 & 195 & 211 & 227 & 243 \\ 11 & 27 & 43 & 59 & 75 & 91 & 107 & 123 & 139 & 155 & 171 & 187 & 203 & 219 & 235 & 251 \\ 7 & 23 & 39 & 55 & 71 & 87 & 103 & 119 & 135 & 151 & 167 & 183 & 199 & 215 & 231 & 247 \\ 15 & 31 & 47 & 63 & 79 & 95 & 111 & 127 & 143 & 159 & 175 & 191 & 207 & 223 & 239 & 255 \end{bmatrix}. \quad (3)$$

3.3.2 The Tweak

The tweak is a block counter to denote the number of blocks that have been absorbed in the **Sponge-F** mode. Concretely, the i -th permutation call of the absorption phase uses

$$\tau = \text{enc}_{64}(i) \parallel 0^{64},$$

where enc_{64} denotes the big-endian 64-bit encoding of the block index.

The operator **AddTweak** denotes the XOR of τ into lane $L_{\ell/2}$, the first lane vector of the capacity C . **AddTweak** is applied at the beginning of every step, so the same tweak τ is added t times within the i -th call of **Wish-P**.

3.3.3 The Round Constants

The round constants **Rcon** form an array of lane vectors. Each **Wish** instance consumes the first $2t$ elements of the array, *i.e.*, 18 for **Wish-512** and 24 for **Wish-1024**.

In step j , **AddConsts^p** denotes the XOR of **Rcon**[p] into lane L_0 for $p \in \{2j, 2j + 1\}$. **Rcon**[0] is chosen as the first 32 digits from the big-endian hexadecimal expansion of the

²Note that this is different from the matrix form for state organization in [Section 3.2](#)

fractional part of π . The update function **LFSR** operates independently on each byte. For a byte $x = (x_7, x_6, x_5, x_4, x_3, x_2, x_1, x_0)$ (with x_7 the most significant bit), it is defined as in **Skinny** [BJK⁺16] and **Deoxys** [JNPS16]:

$$\text{LFSR}(x) = (x_6, x_5, x_4, x_3, x_2, x_1, x_0, x_7 \oplus x_5), \quad (4)$$

i.e., a left rotation by one position with the new least significant bit set to $x_7 \oplus x_5$. When applied to a 128-bit vector, **LFSR** acts on all 16 bytes in parallel. Then the **Rcon** array is generated as $\text{Rcon}[j] = \text{LFSR}(\text{Rcon}[j - 1])$, for $1 \leq j \leq 2t$. The **Rcon** array can be either hard-coded or generated on-the-fly. The full table is shown in [Table 3](#):

Table 3: Round constants of **Wish-P**, listed as 24 128-bit values in hexadecimal.

0	243f6a8885a308d313198a2e03707344	12	660c1ca03df988dee2c8024cf327d450
1	497fd5110b4610a72632155d06e1e788	13	cd1838407bf211bdc4910498e64fa9a0
2	92ffab22168c204e4d652aba0cc2ce11	14	9b307180f7e4227a89230831cc9e5240
3	25fe56452c19419c9acb557418859d22	15	3761e301eec845f513471063993da480
4	4bfcac8a593282393597aae9300b3b45	16	6fc3c602dc918aea268e20c7337b4801
5	96f85815b26505736b2f54d26116778a	17	df878d04b92315d44d1d418f67f79002
6	2df0b02a64cb0ae7d75fa8a5c32cef15	18	bf0f1b0872472aa99a3a821fcfee2104
7	5be06055c99714ceafbe504a8759de2a	19	7e1e3610e58e55523575053e9fdc4308
8	b6c0c1aa932f289d5e7ca0940fb2bd55	20	fd3c6d20ca1daaa46beb0a7d3fb98610
9	6c818354275f513bbcf940291e647aaa	21	fa79db41953a5448d7d614fb7f720d20
10	d90307a84fbea27778f280533cc9f554	22	f4f3b7822b75a890afad28f6ffe51a41
11	b3060e509e7c44eff1e401a67993eaa8	23	e8e66e0557eb50215e5a51ecfeca3482

3.4 Mode: Sponge with Feed-Forward

Wish instantiates the **Sponge-F** mode by Guo *et al.* [GHJ⁺25]. For **Wish-512** and **Wish-1024**, we set $r = c = h = n/2$.

Padding. For $M \in \{0, 1\}^*$, the classical padding appends one bit followed by zeros

$$\text{pd}10^*(M) = M \parallel 10^j$$

is used, where $j = (-|M| - 1) \bmod r$, so that $|\text{pd}10^*(M)|$ is the smallest multiple of r strictly greater than $|M|$. This padding is injective in the sense required by [GHJ⁺25]. The padded message is split into $b = \lfloor |M|/r \rfloor + 1$ blocks $M_0, \dots, M_{b-1}$, each of r bits.

Separation constant. The constant $\theta = 0^{c-1}1 \in \{0, 1\}^c$ marks the final block.

Initialisation.

$$R_0 \leftarrow 0^r, \quad C_0 \leftarrow 0^c, \quad S_0 \leftarrow R_0 \parallel C_0.$$

Absorption. For $i = 0, 1, \dots, b - 1$:

1. Form the permutation input

$$X_i \leftarrow \begin{cases} S_i \oplus (M_i \parallel 0^c), & i < b - 1, \\ S_i \oplus (M_i \parallel \theta), & i = b - 1, \end{cases}$$

and let $F_i \leftarrow \text{right}_c(X_i)$ be the feed-forward value.

$$2. S_{i+1} = R_{i+1} \| C_{i+1} \leftarrow \text{Wish-P}_{\tau_i}(X_i) \oplus (0^r \| F_i).$$

Squeezing. Since $h \leq c$, no additional permutation call is needed. The digest is read directly from the final capacity:

$$H(M) = \text{left}_h(C_b). \quad (5)$$

For Wish-512 and Wish-1024, $h = c$ and the digest is C_b itself. This is the FOL (fixed-output-length) property of Sponge-F [GHJ+25].

3.5 Complete Algorithm

Pseudo-code is given in Algorithms 1 and 2. Algorithm 1 specifies the tweaked permutation Wish-P_τ ; Algorithm 2 specifies the Wish hash function on top of it.

Algorithm 1: The tweaked permutation Wish-P_τ .

Input : state $S \in \{0, 1\}^n$, tweak $\tau \in \{0, 1\}^{128}$

Output : permuted state $\text{Wish-P}_\tau(S) \in \{0, 1\}^n$

```

1  $L_0 \| L_1 \| \dots \| L_{\ell-1} \leftarrow S$                                  $\triangleright$  split state into 128-bit lanes
2 for  $j \leftarrow 0$  to  $t - 1$  do
3    $L_{\ell/2} \leftarrow L_{\ell/2} \oplus \tau$                                  $\triangleright$  AddTweak
4   for  $k \leftarrow 0$  to  $\ell - 1$  do
5      $L_k \leftarrow \text{AESR}(L_k)$                                      $\triangleright$  AESR $^{2j}$ 
6    $L_0 \leftarrow L_0 \oplus \text{Rcon}[2j]$                              $\triangleright$  AddConsts $^{2j}$ 
7   for  $k \leftarrow 0$  to  $\ell - 1$  do
8      $L_k \leftarrow \text{AESR}(L_k)$                                      $\triangleright$  AESR $^{2j+1}$ 
9    $L_0 \leftarrow L_0 \oplus \text{Rcon}[2j + 1]$                          $\triangleright$  AddConsts $^{2j+1}$ 
10   $S \leftarrow \text{Mix}_n(L_0 \| \dots \| L_{\ell-1})$                      $\triangleright$  mixing layer
11 return  $S$ 

```

4 Design Rationale

4.1 Number of AES rounds per step

In this work, we follow the paradigm of *parallel-AES-rounds-then-mix*, in which each step of the primitive applies ρ consecutive AES rounds to all 128-bit lanes in parallel, followed by a state-wide mixing layer for diffusion. Such a paradigm is rich in literature and well-analyzed in a lineage of works. Instances for $\rho = 2$ comprise permutation AESQ [BK14], short-input hashing Haraka [KLMR16], block ciphers Pholkos [BLLS22], ASURA [TSS+24], and Vistrutah [ACCL25]. In contrast, Ghidle [NSA+23] sets $\rho = 3$.

Prior research has been done on the optimal choices of ρ . Shiba, Sakamoto, and Isobe [SSI22] compared constructions with $\rho = 1$ and $\rho = 2$, and concluded that $\rho = 2$ offers better security and higher efficiency. In the design of Vistrutah [ACCL25], the

Algorithm 2: The Wish hash function.

Input : message $M \in \{0, 1\}^*$, $|M| \leq 2^{64} - 1$
Output : digest $H \in \{0, 1\}^h$

// Padding: append 10^* to a multiple of the rate r
1 $b \leftarrow \lfloor |M|/r \rfloor + 1$
2 $M_0 \| M_1 \| \dots \| M_{b-1} \leftarrow \text{pd}(M)$, $M_i \in \{0, 1\}^r$
// Initialisation
3 $R \leftarrow 0^r$; $C \leftarrow 0^c$ $\theta = 1^c$
// Absorption
4 **for** $i \leftarrow 0$ **to** $b - 1$ **do**
5 $\tau_i \leftarrow \text{enc}_{64}(i) \| 0^{64}$ ▷ block counter as 128-bit tweak
6 $R \leftarrow R \oplus M_i$ ▷ absorb message block into rate
7 **if** $i = b - 1$ **then**
8 $C \leftarrow C \oplus \theta$ ▷ mark final block
9 $F \leftarrow C$ ▷ save capacity for feed-forward
10 $R \| C \leftarrow \text{Wish-P}(R \| C, \tau_i)$ ▷ Algorithm 1
11 $C \leftarrow C \oplus F$ ▷ feed-forward
12 **return** $H \leftarrow \text{left}_h(C)$

authors compared candidate designs with $\rho = 2$ and $\rho = 3$ (termed as **Vistrutah.A-2R/3R** in the paper), and concluded that though $\rho = 3$ provides a lower amortized cost per AES round, it takes more AES rounds to achieve the same level of security and active S-boxes, thus a suboptimal choice than $\rho = 2$.

In this work, we also compared the minimum number of active S-boxes for candidates designs with $n \in \{1024, 2048\}$ and $\rho \in \{2, 3, 4\}$ paired with the same mixing strategy. We also take into account of hybrid modes with $\rho = 2$ in the first few and last few steps and $\rho \in \{3, 4\}$ in the middle steps. We finally opt for $\rho = 2$, as it achieves the desired security level with lower total cost of AES rounds and mixing layers.

4.2 Choice of Mix

In the *parallel-AES-rounds-then-mix* paradigm, the mixing layer **Mix** is the sole source of cross-lane diffusion. Its design governs how fast an active difference injected into one lane spreads across the full n -bit state.

We proceed in three steps. First, we state the properties **Mix** should satisfy for resistance to different cryptanalytic attacks. Second, we define the *structure* of **Mix**. To preserve maximum compatibility on modern CPUs, we restrict **Mix** as a sequence of 2-lane interleave operations, wired in stage according to the radix-2 Cooley-Tukey FFT butterfly. This structure does not rely on wide-vector extensions such as **AVX2**, **AVX-512**, or **SVE**, and it benefits from native ISA support on most CPUs, minimal diffusion depth and parallelizability. Third, we exhaust the combinations of *granularity* of interleave operations at each stage and select the candidate that satisfies every stated property while maximizes the active S-box count.

4.2.1 Desired Properties

An ideal Mix should satisfy the following properties.

Property 4.1 (Balanced Lane Spread). *A shuffle σ has the balanced lane spread property if every output lane L_j^{out} receives exactly $16/\ell$ cells from each input lane L_k^{in} , for all $0 \leq j, k < \ell$.*

Property 4.2 (Balanced Column Spread). *A shuffle σ has the balanced column spread property if, for every output column W_k^{out} , the 4 bytes originate from 4 distinct input columns.*

Property 4.3 (Balanced Column Control). *A shuffle σ has the balanced column control property if, for every output column W_k^{out} , the 4 bytes originate from at least 2 distinct input lanes.*

For an ideal candidate Mix with high diffusion power, both $\sigma^{\text{fwd}} = \text{ShiftRows} \circ \text{Mix}$ and $\sigma^{\text{bwd}} = \text{Mix}^{-1}$ need to satisfy all three properties. The reason why we consider also the inverse property is to provide maximal resistance to attacks that follow a middle-out strategy, for instance MITM attacks and rebound attacks.

For $\ell \in \{2, 4\}$, full diffusion of the state can be achieved with 1 full step followed by 1 AES round. For larger internal state sizes of **Wish-P**, i.e., $\ell \in \{8, 16\}$, full diffusion can be achieved with 2 full steps followed by 2 AES rounds in both forward and backward directions.

Property 4.4 (Truncation Barrier). *Let $H \subseteq \{0, \dots, n-1\}$ be the byte positions retained in the digest and $S \setminus H$ the truncated positions. The shuffle Mix has the truncation barrier property if every output column of Mix^{-1} contains at least one byte from $S \setminus H$.*

This barrier ensures that with $S \setminus H$ being truncated off, the bytes retained in H alone cannot be reversely computed through the last **MixColumns** layer of the last step. For **Wish**, where $|H| = |S|/2$, we strengthen this so that $S \setminus H$ controls exactly 2 bytes of every output column of Mix^{-1} , distributing the truncated half evenly across all columns. The property is crucial when considering meet-in-the-middle (MITM) attacks, as the backward computation can only propagate up to the last **MixColumns** output, constraining the position of the matching at the last step.

4.2.2 Structure: A Cooley-Tukey FFT Butterfly of Interleaves

We constrain Mix to a sequence of generic 2-lane interleave operations between a pair of 128-bit lane vectors. The interleave primitives, realized as the **ZIP** family on Arm NEON and the **unpack** family on Intel x86, typically exhibit low latency, high throughput, and are available on all ports of the vector unit, whereas complex cross-lane operations incur substantially higher cost [ACCL25]. Restricting Mix as a sequence of interleave operations therefore maximizes implementation compatibility while keeping the diffusion analysis tractable.

As interleave operations mix 2 lanes at a time, we apply a strategy following the radix-2 Cooley-Tukey FFT butterfly [CT65] to achieve diffusion across all lanes. Let $\ell = 2^k$ be the number of 128-bit lanes, indexed by $i \in \{0, \dots, \ell - 1\}$. Mixing proceeds in $k = \log_2 \ell$ stages; at stage j , each lane i is paired with the lane differing from it in bit j ,

$$\pi_j(i) = i \oplus 2^j, \quad j = 0, 1, \dots, k - 1.$$

For instance, [Figure 1](#) reflects the step function when $\ell = 8$ ($k = 3$), with the mixing pairings as follows:

$$j = 0 : (0, 1) (2, 3) (4, 5) (6, 7), \quad (6)$$

$$j = 1 : (0, 2) (1, 3) (4, 6) (5, 7), \quad (7)$$

$$j = 2 : (0, 4) (1, 5) (2, 6) (3, 7). \quad (8)$$

This schedule achieves full diffusion across all ℓ lanes in $\log_2 \ell$ stages. The partner relations $\{i \oplus 2^j : j = 0, \dots, k-1\}$ generate the additive group $(\mathbb{Z}/2)^k$, so by induction the influence set of each lane doubles at every stage and spans all ℓ lanes after the final one.

4.2.3 Final Selection for Mix

We exhaustively search the space of byte permutations induced by the butterfly mixing structure, parametrized by the granularity of the interleave operation. The interleave admits four granularities for input as a pair of lane vectors, namely 1, 2, 4, and 8 bytes. Within a single stage, all interleave operations use the same granularity. Enumerating every resulting mixing strategy for different ℓ , we retain those satisfying [Properties 4.1](#) to [4.4](#) and, among the survivors, select the candidate with the best active S-box count.

Wish-512. For Wish-512, we have $\ell = 8$, and a total of 3 stages. We search for all $4^3 = 64$ granularity assignments. Of these, 12 satisfy [Properties 4.1](#) to [4.4](#). We adopt the assignment with stage granularities (1, 2, 8).

Wish-1024. For Wish-1024, we have $\ell = 16$, and a total of 4 stages. We search for all $4^4 = 256$ granularity assignments. Of these, 24 satisfy [Properties 4.1](#) to [4.4](#). We finally adopts the assignment with stage granularities (1, 2, 4, 8).

4.3 Tweakable permutations

To obtain a second-preimage security comparable to preimage security, the **Sponge-F** mode in [\[GHJ⁺25\]](#) requires a capacity 64 bits larger than the output length, namely $c = h + 64$. For a fixed permutation width n , this reduces the usable message rate from r to $r - 64$. Instead, **Wish** uses the block counter as the public tweak

$$\tau_i = \text{enc}_{64}(i) \parallel 0^{64},$$

where enc_{64} is the big-endian encoding of the block index. This preserves the full message rate and has several advantages.

- **Clean domain separation for blocks.** By making the permutation family tweakable, we obtain natural domain separation between different absorption positions. Each absorption position is evaluated under a different public tweak. Thus two equal permutation inputs occurring at different block positions are still calls to different members of the tweakable permutation family.
- **Better absorption rate.** The counter used for domain extension does not consume message-rate bits. If a 64-bit counter is added inside every message block, the usable

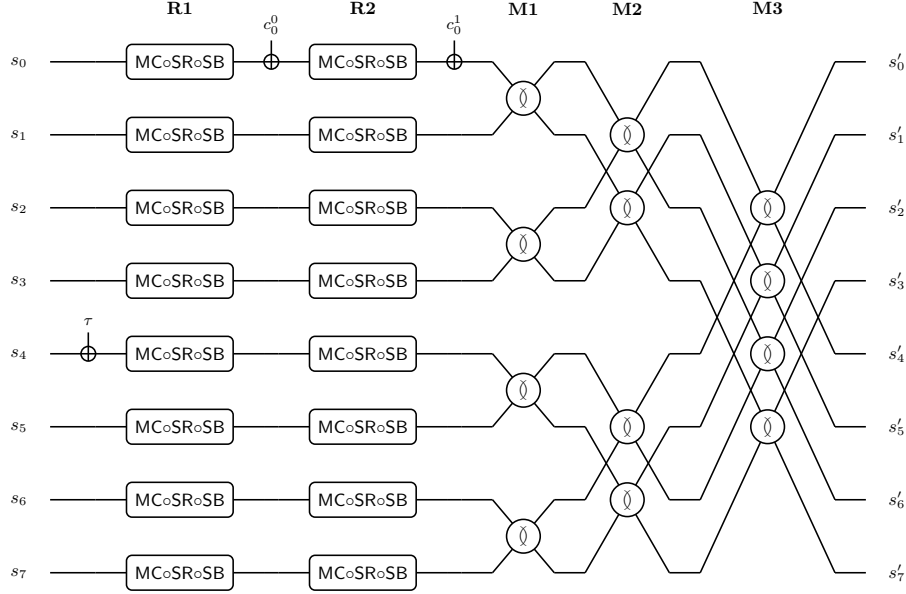


Figure 1: Illustration of Wish-512 step function. τ denotes the block counter, $\bowtie$ denotes the 2-register interleave operation (granularity information omitted).

rate would drop from r to $r - 64$, increasing the number of permutation calls by the factor $r/(r - 64)$ on long messages. For the Wish-512 and Wish-1024, this factor is $512/448$ and $1024/960$, respectively.

- **Almost length-independent second-preimage security.** Following the proof strategy of [BV14], the block-position domain separator prevents the usual multi-target second-preimage attack based on expandable messages. In Wish, this separator is realized naturally by the tweakable permutation family since each block position is evaluated in its own tweak domain. Hence a computation at one block position cannot be explained as the same computation at another position, and the resulting second-preimage bound does not lose a factor proportional to the message length.

We now prove the stronger second-preimage security, following a similar proof strategy as in [BV14].

Theorem 4.1. *Let H be the fixed-output-length hash mode defined in Algorithm 2 with $h \leq c$. Assume that $\{\text{Wish-P}_\tau\}$ is an ideal tweakable permutation family and that the padding is injective. Let $q \leq 2^{n-1}$ be the total number of distinct ideal-permutation evaluations by the adversary, including the evaluations of the challenge message. Then, for every fixed challenge message M ,*

$$\text{Adv}_H^{\text{Spre}}(q) \leq 2q2^{-h} + 2q2^{-c}.$$

For Wish-512 and Wish-1024 we have $h = c$, and hence the advantage is at most $4q2^{-h}$.

Proof. Let $M_0, \dots, M_{b-1}$ be the padded blocks of the challenge message M . Recall the initial state is $S_0 = R_0 \| C_0 = 0^r \| 0^c$, and $S_i = R_i \| C_i$ is the state after i absorption calls. For $0 \leq i < b$, the permutation input is

$$X_i = \begin{cases} S_i \oplus (M_i \| 0^c), & i < b - 1, \\ S_i \oplus (M_i \| \theta), & i = b - 1. \end{cases}$$

Thus $C_{i+1} = \text{right}_c(\text{Wish-P}_{\tau_i}(X_i) \oplus X_i)$, and the final digest is

$$H(M) = \text{left}_h(\text{right}_c(\text{Wish-P}_{\tau_{b-1}}(X_{b-1}) \oplus X_{b-1})).$$

Then we define two bad events.

- **Bad_h**. Let **Bad_h** be the event that some evaluated pair $(i, X) \neq (b-1, X_{b-1})$ satisfies

$$\text{left}_h(\text{right}_c(\text{Wish-P}_{\tau_i}(X) \oplus X)) = H(M).$$

Assuming the ideal tweakable permutation model, for each fresh evaluation, the permutation output is uniform over at least $2^n - q$ remaining n -bit values. At most 2^{n-h} of them satisfy the above h -bit equality. Hence

$$\Pr[\text{Bad}_h] \leq \frac{q 2^{n-h}}{2^n - q}.$$

- Let **Bad_c** be the event that, for some $0 \leq i < b-1$, an evaluated pair (i, X) with $X \neq X_i$ satisfies

$$\text{right}_c(\text{Wish-P}_{\tau_i}(X) \oplus X) = \text{right}_c(\text{Wish-P}_{\tau_i}(X_i) \oplus X_i).$$

The same counting argument gives

$$\Pr[\text{Bad}_c] \leq \frac{q 2^{n-c}}{2^n - q}.$$

Now suppose that the adversary outputs $M' \neq M$ with $H(M') = H(M)$. We show that this implies one of the two bad events above. Let $M'_0, \dots, M'_{b'-1}, S'_i = R'_i \| C'_i$, and X'_i denote the corresponding padded blocks, states, and permutation inputs.

If $(b'-1, X'_{b'-1}) \neq (b-1, X_{b-1})$, then the final evaluation of M' , together with $H(M') = H(M)$, satisfies exactly the defining condition of **Bad_h**.

It remains to consider the case $(b'-1, X'_{b'-1}) = (b-1, X_{b-1})$. Let j be the smallest index such that all permutation inputs from position j are the same, namely

$$X'_k = X_k \quad \text{for every } j \leq k \leq b-1.$$

If $j = 0$, then all permutation inputs are equal. Since both computations start from the same initial state and the update rule is deterministic, all states are equal as well. Hence all padded blocks are equal, and by the injectivity of the padding we obtain $M' = M$, contradicting that M' is a second preimage.

Thus $j > 0$. By the minimality of j ,

$$X'_{j-1} \neq X_{j-1} \quad \text{and} \quad X'_j = X_j.$$

Since $b' = b$, the same separation constant is used at block j in both computations. Therefore $X'_j = X_j$ implies $C'_j = C_j$. Using the update rule for block $j-1$, we get

$$\text{right}_c(\text{Wish-P}_{\tau_{j-1}}(X'_{j-1}) \oplus X'_{j-1}) = \text{right}_c(\text{Wish-P}_{\tau_{j-1}}(X_{j-1}) \oplus X_{j-1}),$$

with $X'_{j-1} \neq X_{j-1}$. Hence **Bad_c** occurs.

Therefore every successful second-preimage attack implies **Bad_h** $\vee$ **Bad_c**. By the union bound,

$$\text{Adv}_H^{\text{Spre}}(q) \leq \Pr[\text{Bad}_h] + \Pr[\text{Bad}_c] \leq \frac{q 2^{n-h}}{2^n - q} + \frac{q 2^{n-c}}{2^n - q}.$$

This proves the theorem. $\square$

5 Security Analysis

5.1 Security Claims (Classical and Quantum)

Table 4 summarizes the classical and quantum security bounds of **Wish** in the ideal tweakable-permutation model and its quantum analogue. The classical preimage, second preimage and collision bounds are obtained by adapting the arguments of Guo *et al.* [GHJ⁺25, Theorems 1 and 5], while the second-preimage bound is further strengthened by Theorem 4.1. The quantum bounds are obtained by applying the quantum lifting theorem of Cojocar *et al.* [CGE⁺24] to the corresponding classical search games.

Table 4: Classical and quantum security bounds of the **Wish** family, expressed in bits.

Instance	Permutation Size n	Classical			Quantum		
		Col.	Pre.	Spre.	qCol.	qPre.	qSpre.
Wish-512	1024	256	512	512	128 [†]	256	256
Wish-1024	2048	512	1024	1024	256 [†]	512	512

(q)Col., (q)Pre., and (q)Spre. denote (quantum) collision, preimage, and second-preimage security, respectively. Quantum collision bound marked with [†] are conservative bounds obtained through the lifting theorem and are not known to be tight.

The quantum collision bounds in Table 4 are the $h/4$ -bit bounds obtained through quantum lifting. In comparison, the BHT algorithm [BHT98] finds a collision for a h -bit random function using $2^{h/3}$ quantum queries, which is known to be optimal. Thus, the table reports the currently justified lifting bounds and does not claim a tight $h/3$ -bit quantum collision bound for **Wish**. For preimage and second-preimage security, quantum lifting gives $h/2$ -bit bounds, matching the generic quantum upper bound.

The classical security bounds claimed for **Wish** meet the requirements of the *Chinese New Generation Commercial Cryptographic Algorithm Competition*. In particular, **Wish-512** provides 256-bit collision resistance and 512-bit preimage and second-preimage resistance, while **Wish-1024** provides 512-bit collision resistance and 1024-bit preimage and second-preimage resistance for messages of length at most $2^{64} - 1$ bits.

5.2 Resistance to Generic Attacks

- **Collision resistance.** Bounded by $h/2$ in the random permutation model [GHJ⁺25, Section 4.2]. The wide capacity ($c \geq h$) ensures no narrow-pipe weakness.
- **Preimage resistance.** Bounded by h in the random permutation model [GHJ⁺25, Section 4.2, Theorem 2]. The feed-forward tightens the reduction compared to standard sponge, yielding a meaningful h -bit bound even in the quantum setting.
- **Second-preimage resistance.** Bounded by h for messages up to 2^{64} blocks. The block counter is used as the public tweak $\tau_i = \text{enc}_{64}(i) \| 0^{64}$, so different absorption positions are evaluated in distinct tweak domains. This gives the almost length-independent bound of Theorem 4.1, avoiding the usual loss proportional to the message length.

- **Length-extension resistance.** The constant θ is a domain separator for the last block, and separates completed digest from intermediate states. The last block of M is processed with θ , whereas the same block would be processed with 0^c if more blocks were appended. Hence $H(M)$ cannot be used to compute $H(M\|M')$.
- **Quantum security.** Following [GHJ⁺25, Section 4.3, Theorem 1], all classical bounds transfer to the quantum random permutation model via the lifting theorem of Cojocaru *et al.* [CGE⁺24].

5.3 Cryptanalysis

This section evaluates the security of **Wish** with a variety of cryptanalysis techniques, including statistical randomness, differential and linear attacks, rebound attacks, algebraic and integral attacks, and meet-in-the-middle attacks.

5.3.1 Statistical Randomness

We assess the output of the **Wish** family with the NIST SP 800-22 empirical randomness suite [Nat10]. For each variant, the test bitstring is formed by concatenating consecutive digests, $H(0)\|H(1)\|\dots$, where the integer counter is encoded as a fixed-width input block and H denotes **Wish-512** or **Wish-1024**. The suite imposes two baseline sizing constraints: a per-sequence length of at least 10^6 bits, required by Maurer’s Universal, Linear Complexity, and the Random Excursion tests, and a sample of at least 55 sequences, required for the second-level uniformity statistic to be meaningful [Nat10, Sec. 2.8, 2.10, 2.14, 4.2]. In our test, we trailed 100 sequences of 2×10^6 bits, each taken sequentially from the generated bitstring. Both **Wish-512** and **Wish-1024** pass every test in the suite at significance level $\alpha = 0.01$.

5.3.2 Differential and Linear Attacks

We assess the resistance of **Wish-P _{τ}** to differential characteristics and linear trails by searching for lower bounds on the number of active AES S-boxes. The search uses a standard word-oriented MILP model [MWGP11], tracking byte activity through **SubBytes**, **ShiftRows**, **MixColumns**, and **Mix**. The resulting bounds for **Wish-512** and **Wish-1024** are reported in Table 5 and Table 6. These bounds support the standard wide-trail security argument. The AES S-box has maximum differential probability 2^{-6} and maximum squared linear correlation 2^{-6} [DR02]; hence, a large number of active S-boxes makes any single differential characteristic or single linear trail negligible for the full permutation. The MILP model is an activity model and does not account for clustering effects.

Table 5: Optimal number of active S-boxes for **Wish-512** permutation.

Rounds	1	2	3	4	5	6	7	8	9	10	11
Bound	1	5	9	25	37	70	86	120	153	165	172
Rounds	12	13	14	15	16	17	18	19	20	21	22
Bound	190	196	220	236	255	303	315	322	340	346	370

Table 6: Optimal number of active S-boxes for Wish-1024 permutation.

Rounds	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Bound	1	5	9	25	41	105	136	200	204	220	224	240	256	320
Rounds	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Bound	336	400	404	420	424	440	456	520	536	600	604	620	624	640

We also consider impossible differential [Knu98, BBS99] and zero-correlation linear attacks [BR14]. Such attacks usually rely on a miss-in-the-middle propagation where the forward and backward parts do not fully mix before they meet. As discussed in Section 4.2, for the relevant lane numbers $\ell \in \{8, 16\}$, full diffusion is reached in both directions after two full steps followed by two AES rounds. Compared with the full Wish-512 and Wish-1024 permutations, which use 9 and 12 steps respectively. We therefore regard the full-round permutations as sufficiently resistant to impossible differentials and zero-correlation linear attacks.

5.3.3 Rebound Attacks

Rebound attacks use an inside-out strategy: an inbound part is solved with the available degrees of freedom, while the outbound conditions are satisfied probabilistically. This technique was introduced for Whirlpool and Grøstl [MRST09], and later refined by the Super-Sbox technique for AES-like permutations [GP10].

For Wish, we follow the Haraka and AESQ analyses [BMS16] and combine them with truncated differential propagation [Knu94]. We fix a 6-round inbound window and use a simple truncated differential model. If A_{in} and A_{out} denote the active bytes at the two inbound boundaries, we estimate the inbound freedom as

$$D_{\text{in}} = 8(A_{\text{in}} + A_{\text{out}}),$$

and accept a candidate when $D_{\text{in}} \geq D_{\text{out}}$. Here D_{out} covers the outbound truncated-differential cost and the all-zero output difference required for a hash collision. We do not impose input-difference restrictions, capacity-zero constraints, or the fixed initial value; hence the relaxed model gives the attacker more freedom than a full hash-level rebound analysis. Table 7 summarizes the search results of our rebound attack models. Overall, the search does not indicate an effective rebound attack on the target rounds. We therefore believe that Wish provides sufficient resistance against rebound attacks.

Table 7: Rebound trails of Wish (by truncated differential probability with $-\log_2$).

Rounds	8	9	10	11	12	13	14	15	16	17	18	19	20
Wish-512	0	0	32	32	128*	128	264*	264*	-	-	-	-	-
Wish-1024	0	0	32	32	128*	128*	392*	392*	608*	608*	-	-	-

*: best found results, not proved optimal.

5.3.4 Algebraic and Integral Attacks

Integral, or square, attacks were introduced together with the Square/Rijndael design strategy [DKR97], and the AES was designed with resistance to such attacks in mind [DR02].

More generally, integral attacks track balanced sums or division-property patterns over structured input sets [KW02, Tod15]. As in Haraka and Vistrutah [KLMR16, ACCL25], their relevance for AES-based hash permutations depends on diffusion speed, the number of AES S-boxes involved, and whether a distinguisher can be turned into a meaningful hash attack. In **Wish**, full diffusion is reached after a small number of steps as discussed in Section 4.2, while the target permutations use substantially more rounds. Although we have not carried out a dedicated integral search in this version, the AES-based design experience, the diffusion margin of **Wish**, and the hash-level security target suggest that integral attacks do not pose a practical threat to the claimed security of **Wish**.

Algebraic attacks on AES exploit compact polynomial descriptions of the AES S-box and round function. The XL/XSL line of work and the BES embedding suggested that the algebraic structure of Rijndael might be exploitable against AES [CKPS00, MR02, CP02]. Subsequent analyses of XSL and BES clarified that these approaches do not give a practical threat to full AES security [CL05, LK07]. Since **Wish** follows the AES wide-trail strategy, uses the same AES S-box nonlinearity, and rapidly activates many S-boxes through intra-lane and inter-lane diffusion, algebraic attacks are not expected to be a stronger attack vector against **Wish**.

5.3.5 Meet-in-the-Middle Attacks

Meet-in-the-Middle (MITM) attacks are central to hash function cryptanalysis. It dates back to the test-of-time work by Sasaki and Aoki at Asiacrypt 2008 [SA08]. At Eurocrypt 2021, Bao *et al.* built the first MILP-based search tool for MITM attacks on AES-like hashing with Merkle-Damgård constructions [BDG⁺21]. The tool was later furnished with a number of techniques [DHS⁺21, BGST22, CGL⁺24, CGL⁺25], and extended to sponge constructions [QHD⁺23, DZQ⁺24]. In this report, we follow the lineage of this tool to search for MITM attacks on **Wish**.

We define attacks on truncated **Wish-P** as follows. Let $Y\|Z = \text{Wish-P}(X\|C)$, where X, Y denote the rate part of the input and output of **Wish-P**, C denote the initial value or some fixed constants, and Z denote the final digest. A preimage attack on truncated **Wish-P** succeeds if, given Z , an adversary finds a pair of (X, C) within 2^h evaluations of **Wish-P**, such that $\text{left}_h \text{Wish-P}(X\|C) = Z$. A collision attack on truncated **Wish-P** succeeds if, an adversary finds (X, C) and (X', C') within $2^{h/2}$ evaluations of **Wish-P**, such that $\text{left}_h \text{Wish-P}(X\|C) = \text{left}_h \text{Wish-P}(X'\|C')$.

We limit the analysis in this work to the one-block attack setting, in which C is fixed to $IV + \theta$, and the adversary needs to find X satisfy with both Z and C . It is clear that an attack on truncated **Wish-P** is strictly stronger than a one-block attack on **Wish**. And an attack on truncated **Wish-P** does not translate into any meaningful one-block attack on **Wish**. Hence, we search for attacks on truncated **Wish-P** instead of one-block attacks on **Wish** with the intent to estimate an upper bound on attack rounds. Also, due to Property 4.4 of **Mix**, the search space for MITM preimage and collision attack coincide, as backward computation is gated at the last **MixColumns** layer. Thus, any valid attacks must meet at the last round, which has leads to a smaller search space compared to previous attacks on AES-like hashing. We then adapted and applied the search tool to find MITM attacks on truncated **Wish-P**. And preliminary results show the MITM attacks reaches 8 rounds for both **Wish-P-1024** and **Wish-P-2048**. For multi-block attack setting, we note that it is possible to obtain stronger attacks by leveraging the extra degrees of freedom and leave it for future analysis.

6 Performance

6.1 Software Performance

We measure the software performance using the evaluation framework provided by NICCS³ on both x86 and ARM platforms. Each instance is benchmarked at eight input lengths,

$$|M| \in \{32, 128, 512, 1024, 4096, 8192, 16384, 65536\} \text{ bytes,}$$

with results averaged over 10,000 trials. The baseline for compiler flags are chosen following the NICCS guide for self-evaluation. In the optimized implementations, we make use of the hardware support for AES that are common on both platforms to maximize performance (*i.e.*, enable flag `-maes` on x86-64 and `-march=...+aes` on AArch64). We also note that our implementation does not rely on the AVX2/SVE support on x86-64/AArch64. Enabling those flags leads to a small turbulence of the performance (either positively or negatively, to a small extent). The benchmarks are shown in Tables 8 to 11.

Table 8: x86-64 performance of Wish. Processor: Intel Xeon Platinum 8358; clock: 2.60 GHz; RAM: 256 GB ECC DDR4-3200; OS: Ubuntu 22.04.5 LTS, kernel 6.8.0-124-generic (x86_64); compiler: gcc 11.4; build tool: CMake 3.22.1; benchmark tool: https://github.com/openHiTLS/ngcc_bench; start-end: 2026-06-26 21:06:15–21:13:14 (UTC+8).

Metric	Instance	Input length (bytes)							
		32	128	512	1024	4096	8192	16384	65536
Cpb	Wish-512 [†]	9.56	5.28	3.64	3.39	3.13	3.09	3.08	3.07
	Wish-1024 [‡]	22.50	9.72	5.82	5.13	4.58	4.47	4.42	4.38
Mbps	Wish-512 [†]	1273	2978	5096	5758	6530	6675	6726	6773
	Wish-1024 [‡]	710	1812	3318	3885	4490	4630	4694	4748
Avg time (μ s)	Wish-512 [†]	0.15	0.29	0.75	1.37	4.97	9.77	19.44	77.35
	Wish-1024 [‡]	0.31	0.51	1.18	2.06	7.25	14.10	27.86	110.34

[†] Best performance found with `-mavx2` and `-maes` enabled.

[‡] Best performance found with `-mavx2` disabled and `-maes` enabled.

Table 9: x86-64 memory profile of Wish on AMD EPYC 7763 (Zen 3). RSS growth measured over the stability run; both builds reported STABLE.

Metric	Wish-512	Wish-1024
Static memory (bytes)	2109440	2109440
Peak memory (bytes)	4206592	4206592

³https://github.com/openHiTLS/ngcc_bench

Table 10: AArch64 performance of **Wish**. Processor: Rockchip RK3588 (Cortex-A76); clock: 2.0GHz; RAM: 16G; OS: ubuntu 22.04, kernel 5.10.160 (aarch64); compiler: gcc 9.4; build tool: CMake 3.16.3; benchmark tool: https://github.com/openHiTLS/ngcc_bench; start-end: 2026-06-26 21:53:09–21:54:47 (UTC+8).

Metric	Instance	Input length (bytes)							
		32	128	512	1024	4096	8192	16384	65536
Cpb	Wish-512 [†]	41.30	14.53	6.83	5.54	4.57	4.41	4.32	4.27
	Wish-1024 [†]	58.08	20.58	9.70	7.90	6.56	6.33	6.22	6.14
Mbps	Wish-512 [†]	106	383	1173	1794	2964	3326	3543	3720
	Wish-1024 [†]	95	335	970	1420	2167	2379	2499	2598
Avg time (μ s)	Wish-512 [†]	2.31	2.56	3.38	4.46	10.94	19.59	36.89	140.81
	Wish-1024 [†]	2.58	2.95	4.11	5.66	15.01	27.44	52.35	201.72

[†] Built with `-sve` disabled and `-aes` enabled.

Table 11: AArch64 memory profile of **Wish** on Rockchip RK3588 (Cortex-A76). RSS growth measured over the stability run; both builds reported **STABLE**.

Metric	Wish-512	Wish-1024
Static memory (bytes)	135168	135168
Peak memory (bytes)	135168	135168

6.2 Hardware Performance

We give a rough estimate of the hardware cost of **Wish** in this report. We use one AES round function (with round key addition) as an atomic unit for normalization. Table 12 reports the normalized area $\hat{A}_R$ and serial delay $\hat{D}$ of the round-based implementation of **Wish** with comparisons to AES and AES-based designs. Concrete ASIC or FPGA claims require further dedicated RTL implementation and synthesis.

Table 12: Comparative hardware cost estimate of round-based implementations of AES-based primitives. The area and serial delay are reported in equivalent numbers of single AES round.

	Primitive	Size	Rounds	$\hat{A}_R$	$\hat{D}$
Reference	Single AES round	128	1	1.00	1.00
Block ciphers [†]	AES-128	128	10	1.27	10.52
	Rijndael-256-256	256	14	2.60	14.77
	Vistrutah-256 full	256	14	2.08	14.77
	Vistrutah-512 full	512	18	4.16	19.03
Hash Functions [‡]	Haraka-256 v2	256	10	2.00	10.00
	Haraka-512 v2	512	10	4.00	10.00
	Wish-512	1024	18	8.00	18.00
	Wish-1024	2048	24	16.00	24.00

[†] For block ciphers, the estimates originates from [ACCL25] and includes the key schedules. The data are normalized to round-based implementations.

[‡] For hash functions, the estimate ignores mode-level XORs and count only the core operations.

References

- [ACCL25] Roberto Avanzi, Avik Chakraborti, Bishwajit Chakraborty, and Eik List. The large block cipher vistrutah. *IACR Trans. Symmetric Cryptol.*, 2025(3):1–150, 2025. URL: <https://doi.org/10.46586/tosc.v2025.i3.1-150>, doi:10.46586/TOSC.V2025.I3.1-150.
- [BBS99] Eli Biham, Alex Biryukov, and Adi Shamir. Cryptanalysis of Skipjack Reduced to 31 Rounds Using Impossible Differentials. In *Advances in Cryptology - EUROCRYPT '99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding*, pages 12–23, 1999.
- [BDG⁺21] Zhenzhen Bao, Xiaoyang Dong, Jian Guo, Zheng Li, Danping Shi, Siwei Sun, and Xiaoyun Wang. Automatic Search of Meet-in-the-Middle Preimage Attacks on AES-like Hashing. In *EUROCRYPT I*, pages 771–804, 2021.
- [BDPA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the Indifferentiability of the Sponge Construction. In *EUROCRYPT*, pages 181–197, 2008.
- [BGST22] Zhenzhen Bao, Jian Guo, Danping Shi, and Yi Tu. Superposition Meet-in-the-Middle Attacks: Updates on Fundamental Security of AES-like Hashing. In *CRYPTO I*, volume 13507, pages 64–93, 2022.
- [BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum Cryptanalysis of Hash and Claw-Free Functions. In *Latin American Theoretical Informatics (LATIN 1998)*, volume 1380 of *Lecture Notes in Computer Science*, pages 163–169. Springer, 1998.
- [BJK⁺16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY Family of Block Ciphers and Its Low-Latency Variant MANTIS. In *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II*, volume 9815, pages 123–153. Springer, 2016.
- [BK14] Alex Biryukov and Dmitry Khovratovich. PAEQ: parallelizable permutation-based authenticated encryption. In *Information Security - 17th International Conference, ISC 2014, Hong Kong, China, October 12-14, 2014, Proceedings*, volume 8783 of *Lecture Notes in Computer Science*, pages 72–89. Springer, 2014. doi:10.1007/978-3-319-13257-0_5.
- [BLLS22] Jannis Bossert, Eik List, Stefan Lucks, and Sebastian Schmitz. Pholkos - efficient large-state tweakable block ciphers from the AES round function. In *Topics in Cryptology - CT-RSA 2022, Cryptographers' Track at the RSA Conference 2022, Virtual Event, March 1-2, 2022, Proceedings*, volume 13161 of *Lecture Notes in Computer Science*, pages 511–536. Springer, 2022. doi:10.1007/978-3-030-95312-6_21.

- [BMS16] Nasour Bagheri, Florian Mendel, and Yu Sasaki. Improved rebound attacks on AESQ: core permutation of CAESAR candidate PAEQ. In Joseph K. Liu and Ron Steinfeld, editors, *Information Security and Privacy - 21st Australasian Conference, ACISP 2016, Melbourne, VIC, Australia, July 4-6, 2016, Proceedings, Part II*, Lecture Notes in Computer Science, pages 301–316. Springer, 2016. doi:[10.1007/978-3-319-40367-0_19](https://doi.org/10.1007/978-3-319-40367-0_19).
- [BR14] Andrey Bogdanov and Vincent Rijmen. Linear hulls with correlation zero and linear cryptanalysis of block ciphers. *Des. Codes Cryptogr.*, 70(3):369–383, 2014.
- [BV14] Charles Bouillaguet and Bastien Vayssière. Provable second preimage resistance revisited. In Tanja Lange, Kristin Lauter, and Petr Lisoněk, editors, *Selected Areas in Cryptography – SAC 2013*, pages 513–532, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.
- [CGE⁺24] Almudena Cao Piñero, Juanrui Guo, Edward Eaton, et al. A lifting theorem for evasiveness, with applications to quantum security. Cryptology ePrint Archive, Paper 2024/1839, 2024. URL: <https://eprint.iacr.org/2024/1839>.
- [CGL⁺24] Shiyao Chen, Jian Guo, Eik List, Danping Shi, and Tianyu Zhang. Diving deep into the preimage security of aes-like hashing. In *EUROCRYPT (1)*, volume 14651 of *Lecture Notes in Computer Science*, pages 398–426. Springer, 2024.
- [CGL⁺25] Shiyao Chen, Jian Guo, Eik List, Danping Shi, and Tianyu Zhang. Scrutinizing the Security of AES-based Hashing and One-way Functions. In *ASIACRYPT*, 2025.
- [CKPS00] Nicolas T. Courtois, Alexander Klimov, Jacques Patarin, and Adi Shamir. Efficient Algorithms for Solving Overdefined Systems of Multivariate Polynomial Equations. In *Advances in Cryptology - EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 392–407. Springer, 2000.
- [CL05] Carlos Cid and Gaëtan Leurent. An Analysis of the XSL Algorithm. In *Advances in Cryptology - ASIACRYPT 2005*, volume 3788 of *Lecture Notes in Computer Science*, pages 333–352. Springer, 2005.
- [CP02] Nicolas T. Courtois and Josef Pieprzyk. Cryptanalysis of Block Ciphers with Overdefined Systems of Equations. In *Advances in Cryptology - ASIACRYPT 2002*, volume 2501 of *Lecture Notes in Computer Science*, pages 267–287. Springer, 2002.
- [CT65] James W. Cooley and John W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Math. Comput.*, 19:297–301, 1965. doi:[10.1090/S0025-5718-1965-0178586-1](https://doi.org/10.1090/S0025-5718-1965-0178586-1).
- [DHS⁺21] Xiaoyang Dong, Jialiang Hua, Siwei Sun, Zheng Li, Xiaoyun Wang, and Lei Hu. Meet-in-the-Middle Attacks Revisited: Key-Recovery, Collision, and Preimage Attacks. In *CRYPTO III*, volume 12827, pages 278–308, 2021.

- [DKR97] Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. The block cipher Square. In *Fast Software Encryption, 4th International Workshop, FSE '97, Haifa, Israel, January 20-22, 1997, Proceedings*, volume 1267 of *Lecture Notes in Computer Science*, pages 149–165. Springer, 1997. doi:[10.1007/BFb0052343](https://doi.org/10.1007/BFb0052343).
- [DR02] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2002. doi:[10.1007/978-3-662-04722-4](https://doi.org/10.1007/978-3-662-04722-4).
- [DZQ⁺24] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic MitM Attack Frameworks on Sponge Constructions. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, pages 3–37, 2024.
- [GHJ⁺25] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Permutation-Based Hash from Non-Idealized Assumptions: Adding Feed-Forward to Sponge. Cryptology ePrint Archive, Paper 2025/1006, 2025. URL: <https://eprint.iacr.org/2025/1006>.
- [GP10] Henri Gilbert and Thomas Peyrin. Super-Sbox Cryptanalysis: Improved Attacks for AES-Like Permutations. In *FSE*, volume 6147, pages 365–383, 2010.
- [JNPS16] Jérémy Jean, Ivica Nikolić, Thomas Peyrin, and Yannick Seurin. Submission to CAESAR : Deoxys v1.41, October 2016. <http://competitions.cr.yp.to/round3/deoxysv141.pdf>.
- [KLMR16] Stefan Kölbl, Martin M. Lauridsen, Florian Mendel, and Christian Rechberger. Haraka v2 - efficient short-input hashing for post-quantum applications. *IACR Transactions on Symmetric Cryptology*, 2016(2):1–29, 2016. doi:[10.13154/tosc.v2016.i2.1-29](https://doi.org/10.13154/tosc.v2016.i2.1-29).
- [Knu94] Lars R. Knudsen. Truncated and Higher Order Differentials. In *Fast Software Encryption, Second International Workshop, Leuven, Belgium, December 14-16, 1994, Proceedings*, volume 1008, pages 196–211, 1994.
- [Knu98] Lars R. Knudsen. DEAL - A 128-bit Block Cipher. In *NIST AES Proposal*, 1998.
- [KW02] Lars R. Knudsen and David A. Wagner. Integral Cryptanalysis. In *Fast Software Encryption, 9th International Workshop, FSE 2002*, pages 112–127, 2002.
- [LK07] Chu-Wee Lim and Khoongming Khoo. An Analysis of XSL Applied to BES. In *Fast Software Encryption, 14th International Workshop, FSE 2007*, volume 4593 of *Lecture Notes in Computer Science*, pages 242–253. Springer, 2007.
- [MR02] Sean Murphy and Matthew J. B. Robshaw. Essential Algebraic Structure within the AES. In *Advances in Cryptology - CRYPTO 2002*, volume 2442 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 2002.

- [MRST09] Florian Mendel, Christian Rechberger, Martin Schl  ffer, and S  ren S. Thomsen. The Rebound Attack: Cryptanalysis of Reduced Whirlpool and Gr  stl. In *FSE*, volume 5665, pages 260–276, 2009.
- [MWGP11] Nicky Mouha, Qingju Wang, Dawu Gu, and Bart Preneel. Differential and linear cryptanalysis using mixed-integer linear programming. In *International Conference on Information Security and Cryptology*, volume 7537, pages 57–76. Springer, 2011.
- [Nat10] National Institute of Standards and Technology. A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications, SP 800-22. <https://csrc.nist.gov/pubs/sp/800/22/r1/final>, 2010.
- [NSA⁺23] Motoki Nakahashi, Rentaro Shiba, Ravi Anand, Mostafizar Rahman, Kosei Sakamoto, Fukang Liu, and Takanori Isobe. Ghidle: Efficient large-state block ciphers for post-quantum security. In *Information Security and Privacy - 28th Australasian Conference, ACISP 2023, Brisbane, QLD, Australia, July 5-7, 2023, Proceedings*, volume 13915 of *Lecture Notes in Computer Science*, pages 403–430. Springer, 2023. doi:10.1007/978-3-031-35486-1_18.
- [QHD⁺23] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-Middle Preimage Attacks on Sponge-Based Hashing. In *EUROCRYPT IV*, pages 158–188, 2023.
- [SA08] Yu Sasaki and Kazumaro Aoki. Preimage Attacks on 3, 4, and 5-Pass HAVAL. In *ASIACRYPT*, volume 5350, pages 253–271, 2008.
- [SSI22] Rentaro Shiba, Kosei Sakamoto, and Takanori Isobe. Efficient constructions for large-state block ciphers based on AES new instructions. *IET Information Security*, 16(3):145–160, 2022. doi:10.1049/ise2.12053.
- [Tod15] Yosuke Todo. Structural Evaluation by Generalized Integral Property. In *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, pages 287–314, 2015.
- [TSS⁺24] Atsushi Tanaka, Rentaro Shiba, Kosei Sakamoto, Mostafizar Rahman, Takuro Shiraya, and Takanori Isobe. ASURA: an efficient large-state tweakable block cipher for ARM environment. In *Progress in Cryptology - INDOCRYPT 2024 - 25th International Conference on Cryptology in India, Chennai, India, December 18-21, 2024, Proceedings, Part I*, volume 15495 of *Lecture Notes in Computer Science*, pages 143–164. Springer, 2024. doi:10.1007/978-3-031-80308-6_7.